

Gestione degli errori

Errori comuni

Gli errori Java più frequenti quando si inizia e come leggerli.

Gli errori fanno parte del lavoro

Quando impari Java, vedrai molti messaggi di errore. Non significano che stai sbagliando percorso.

Un errore è un'informazione: Java ti sta dicendo che qualcosa non torna.

La cosa importante è imparare a leggere il messaggio con calma.

Errori di compilazione

Un errore di compilazione succede quando `javac` non riesce a trasformare il file `.java` in `.class`.

Esempio:

```
public class Errore {  
    public static void main(String[] args) {  
        System.out.println("Ciao")  
    }  
}
```

Manca il punto e virgola.

Java potrebbe mostrare un messaggio simile:

```
error: ';' expected
```

Correzione:

```
System.out.println("Ciao");
```

Nome della classe e nome del file

Se scrivi:

```
public class Saluto {  
}
```

il file deve chiamarsi:

```
Saluto.java
```

Se il file ha un altro nome, Java segnala errore.

Variabile non trovata

```
public class Esempio {  
    public static void main(String[] args) {  
        int eta = 20;  
        System.out.println(anni);  
    }  
}
```

anni non esiste.

Java segnala che non trova quel simbolo.

Correzione:

```
System.out.println(eta);
```

Controlla spesso nomi scritti in modo diverso: `eta` , `Eta` e `età` non sono la stessa cosa.

Tipi incompatibili

```
int eta = "venti";
```

`int` richiede un numero intero. `"venti"` è una stringa.

Correzione:

```
int eta = 20;
```

Oppure:

```
String eta = "venti";
```

Dipende da cosa ti serve davvero.

Errori a runtime

Un errore a runtime succede mentre il programma è già in esecuzione.

Esempio:

```
int[] numeri = {10, 20, 30};  
System.out.println(numeri[3]);
```

Il codice compila, ma durante l'esecuzione Java segnala:

```
ArrayIndexOutOfBoundsException
```

L'array ha indici `0` , `1` , `2` . L'indice `3` non esiste.

Leggere lo stack trace

Quando un programma fallisce, Java mostra spesso uno **stack trace**.

Sembra lungo, ma all'inizio cerca soprattutto:

- il nome dell'errore
- il file
- il numero di riga

Esempio:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException  
    at Esempio.main(Esempio.java:4)
```

La parte utile è:

```
Esempio.java:4
```

Vai alla riga 4 e controlla cosa succede lì.

Metodo di controllo

Quando trovi un errore:

1. leggi la prima riga del messaggio
2. cerca il numero di riga
3. controlla nomi, parentesi e punto e virgola
4. verifica i tipi delle variabili
5. se serve, stampa valori intermedi

Gli errori diventano meno spaventosi quando li tratti come indizi.

Debugging

Come trovare problemi nel codice usando stampe, debugger e ragionamento.

Cosa significa fare debugging

Fare debugging significa cercare e correggere un problema nel codice.

Non è solo "togliere errori". E capire cosa il programma sta facendo davvero, passo dopo passo.

Leggere prima il messaggio

Quando Java mostra un errore, non partire subito a cambiare codice a caso.

Prima cerca:

- tipo di errore
- file
- numero di riga
- valore o nome citato nel messaggio

Esempio:

```
Exception in thread "main" java.lang.NumberFormatException  
    at Esempio.main(Esempio.java:8)
```

Vai alla riga 8 e chiediti: sto convertendo testo in numero? Il testo contiene davvero un numero?

Usare stampe temporanee

Un modo semplice per capire cosa succede e stampare valori.

```
int prezzo = 10;
int quantita = 3;
int totale = prezzo * quantita;

System.out.println("prezzo = " + prezzo);
System.out.println("quantita = " + quantita);
System.out.println("totale = " + totale);
```

Queste stampe sono temporanee. Servono durante il controllo.

Quando hai trovato il problema, rimuovile o trasformale in output utile.

Isolare il problema

Se un programma è lungo, prova a restringere il punto del problema.

Chiediti:

1. il valore entra corretto?
2. cambia nel punto giusto?
3. viene passato al metodo corretto?
4. l'errore compare prima o dopo questa riga?

Puoi commentare temporaneamente alcune parti o creare un esempio più piccolo.

Usare il debugger

Molti editor, come Visual Studio Code o IntelliJ IDEA, hanno un debugger.

Il debugger permette di:

- fermare il programma su una riga
- eseguire una riga alla volta
- guardare il valore delle variabili
- capire quale ramo di un `if` viene eseguito

Il punto in cui fermi il programma si chiama **breakpoint**.

Un esempio pratico

Immagina questo codice:

```
int[] voti = {7, 8, 6};
int somma = 0;

for (int i = 0; i <= voti.length; i++) {
    somma = somma + voti[i];
}

System.out.println(somma);
```

Il programma fallisce. Il problema e nella condizione:

```
i <= voti.length
```

L'ultimo indice valido e `voti.length - 1`.

Correzione:

```
for (int i = 0; i < voti.length; i++) {
    somma = somma + voti[i];
}
```

Metodo semplice

Quando non capisci un errore:

1. riprodurlo
2. leggi il messaggio
3. trova la riga
4. stampa o osserva le variabili vicine
5. fai una modifica piccola
6. ricontrolla

Il debugging richiede pazienza. Una modifica piccola alla volta ti evita di creare nuovi problemi mentre cerchi quello iniziale.

Eccezioni

Come gestire situazioni impreviste con try, catch e finally.

Cosa e un'eccezione

Un'eccezione e un problema che avviene mentre il programma e in esecuzione.

Esempi:

- un file non esiste
- l'utente inserisce testo al posto di un numero
- un array viene letto fuori dai suoi limiti

Java usa le eccezioni per segnalare questi casi.

try e catch

Con `try catch` puoi provare un'operazione e gestire l'errore se succede.

```
try {  
    int numero = Integer.parseInt("abc");  
    System.out.println(numero);  
} catch (NumberFormatException e) {  
    System.out.println("Non e un numero valido.");  
}
```

Output:

```
Non e un numero valido.
```

`Integer.parseInt("abc")` fallisce perche `"abc"` non e un numero.

Un esempio con input

```
public class LetturaNumero {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Numero: ");
        String testo = scanner.nextLine();

        try {
            int numero = Integer.parseInt(testo);
            System.out.println("Doppio: " + (numero * 2));
        } catch (NumberFormatException e) {
            System.out.println("Devi inserire un numero intero.");
        }

        scanner.close();
    }
}
```

Se l'utente scrive `12`, il programma calcola il doppio.

Se scrive `ciao`, il programma mostra un messaggio chiaro invece di interrompersi.

finally

`finally` contiene codice che viene eseguito comunque, sia se tutto va bene sia se c'è un errore.

```
try {
    System.out.println("Provo a fare qualcosa");
} catch (Exception e) {
    System.out.println("Qualcosa è andato storto");
} finally {
    System.out.println("Questa riga viene eseguita sempre");
}
```

È utile per chiudere risorse, come file o connessioni.

Eccezioni controllate e non controllate

Java distingue due famiglie importanti.

Le **eccezioni controllate** devono essere gestite o dichiarate. Un esempio comune è `IOException`, usata quando lavori con file.

Le **eccezioni non controllate** possono avvenire a runtime e Java non ti obbliga sempre a gestirle. Esempi:

- `NumberFormatException`
- `ArrayIndexOutOfBoundsException`
- `NullPointerException`

Per iniziare non devi memorizzare tutte le categorie. Ti basta sapere che alcune operazioni, come i file, obbligano a pensare agli errori.

Non catturare tutto senza motivo

Potresti scrivere:

```
catch (Exception e) {  
    System.out.println("Errore");  
}
```

Funziona, ma è molto generico.

Quando puoi, cattura l'eccezione specifica:

```
catch (NumberFormatException e) {  
    System.out.println("Numero non valido.");  
}
```

Il messaggio è più chiaro e il codice è più facile da controllare.

Regola pratica

Usa le eccezioni per gestire problemi che possono capitare anche in un programma scritto bene: input sbagliato, file mancanti, dati non validi.

Non usarle per nascondere errori che dovresti correggere nel codice.